

TABLE DES MATIÈRES

1. INTRODUCTION¹

3. CONCEPTION

- 3.1. Diagramme de classes
- 3.2. Description des classes
- 3.3. Structure de la liste chaînée

5. TESTS ET RÉSULTATS

- 5.1. Captures d'écran
- 5.2. Tests des fonctionnalités

2. ANALYSE DU PROJET

- 2.1. Présentation du problème
- 2.2. Fonctionnalités demandées
- 2.3. Choix des outils

4. RÉALISATION

- 4.1. Organisation du code
- 4.2. Les structures de données
- 4.3. Les interfaces graphiques

6. CONCLUSION⁶⁶

1. INTRODUCTION

Dans le cadre de l'UE Structures de Données et Implémentation (TIC233), nous avons développé un projet de gestion des emprunts pour une bibliothèque. Ce projet vise à appliquer les concepts fondamentaux étudiés en cours, tels que les listes doublement chaînées, la manipulation des pointeurs, et les algorithmes de recherche et de tri.

Une bibliothèque moderne est confrontée à la tâche complexe de gérer un flux constant d'emprunts et de retours de livres. Notre système informatique est conçu pour simplifier cette gestion, offrant un suivi efficace et précis de toutes les transactions d'emprunt, ce qui est crucial pour maintenir l'ordre et l'accessibilité de la collection.

L'objectif principal de ce projet est de créer une application robuste et intuitive capable de gérer les opérations essentielles d'une bibliothèque. Les fonctionnalités clés incluent :

1 Enregistrer les emprunts de livres

Permettre l'ajout rapide et précis de nouveaux emprunts au système.

2 Gérer les retours de livres

Faciliter le processus de retour, mettant à jour la disponibilité des livres.

3 Rechercher un livre dans l'historique

Offrir une capacité de recherche efficace pour localiser des livres spécifiques ou des informations d'emprunt.

4 Naviguer dans l'historique des emprunts

Permettre une exploration fluide des enregistrements d'emprunt passés et présents.

5 Compter le nombre d'emprunts

Fournir des statistiques instantanées sur le volume d'emprunts actifs.

6 Trier les emprunts

Organiser les données d'emprunt selon divers critères pour une meilleure analyse.

Pour atteindre ces objectifs, nous avons choisi d'utiliser le langage **C++** pour sa performance et sa flexibilité, la bibliothèque **Qt** pour le développement d'une interface graphique conviviale, et une **liste doublement chaînée** comme structure de données centrale pour une gestion optimale des emprunts.

2. ANALYSE DU PROJET

2.1. Présentation du problème

Le défi central de ce projet est de concevoir un système capable de gérer efficacement les emprunts au sein d'une bibliothèque, qu'elle soit de petite ou de grande taille. La problématique se résume à : **Comment gérer efficacement les emprunts d'une bibliothèque qui possède de nombreux livres et adhérents ?**

Pour y répondre, le système doit impérativement permettre :

- L'ajout rapide de nouveaux emprunts.
- La localisation aisée d'un livre emprunté.
- Une navigation intuitive dans l'historique des transactions.
- L'obtention rapide du nombre total d'emprunts.

La solution retenue est l'utilisation d'une **liste doublement chaînée**, structure de données idéale pour la navigation bidirectionnelle (précédent/suivant), essentielle pour l'historique des emprunts.

2.2. Fonctionnalités demandées

Le cahier des charges spécifie six fonctionnalités clés à implémenter :

1. Enregistrement d'emprunt Ajout d'un nouvel emprunt en tête de liste (le plus récent) lorsqu'un membre emprunte un livre.	2. Retour de livre Suppression de l'emprunt de la liste lorsque le livre est retourné.
3. Recherche de livre Vérification de la disponibilité d'un livre en recherchant s'il est actuellement emprunté.	4. Navigation dans l'historique Déplacement entre les emprunts précédents et suivants grâce aux pointeurs de la liste.
5. Calcul du nombre d'emprunts Détermination du nombre total d'emprunts actifs dans le système.	6. Tri du catalogue Organisation des emprunts par date, par membre ou par titre de livre pour une meilleure gestion.

2.3. Choix des outils

Pour concrétiser ce projet, nous avons sélectionné des outils spécifiques :

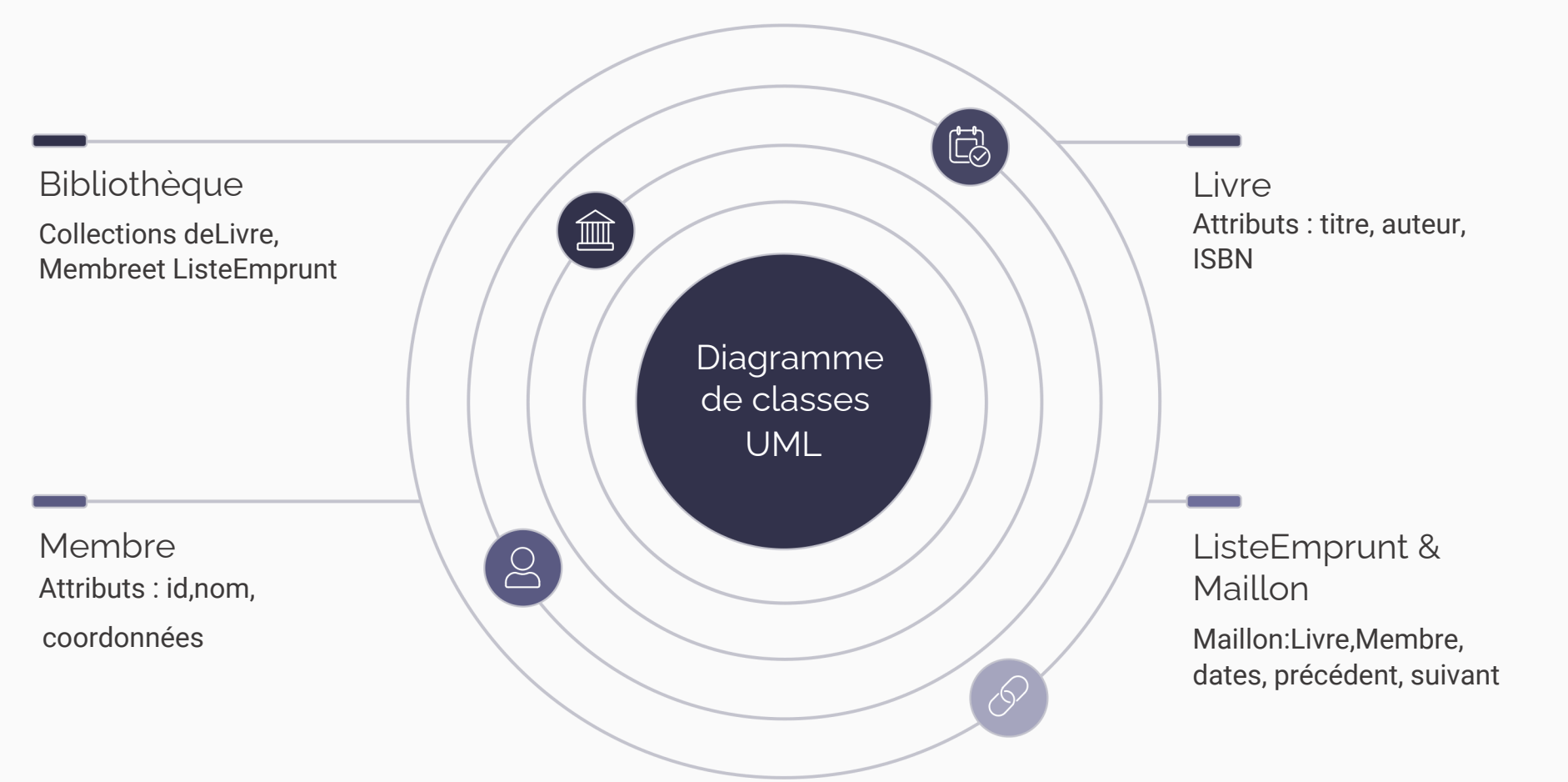
Langage de programmation : C++ • Imposé par le cahier des charges. • Permet la manipulation directe de la mémoire (pointeurs). • Supporte la programmation orientée objet.	Framework graphique : Qt • Facilite la création d'interfaces graphiques multiplateformes. • Qt Designer offre une conception visuelle de l'interface. • Excellente intégration avec C++.	Structure de données : Liste doublement chaînée Avantages : • Navigation bidirectionnelle (précédent/suivant). • Ajout rapide en tête de liste. • Gestion dynamique de la mémoire. • • Inconvénients : • Recherche plus lente qu'un tableau. • Utilisation accrue de la mémoire (deux pointeurs par élément).
--	--	--

La liste doublement chaînée est particulièrement adaptée à notre projet en raison de la nécessité fréquente de naviguer dans l'historique des emprunts.

3. CONCEPTION

3.1. Diagramme de classes

Le diagramme de classes ci-dessous illustre l'architecture de notre application, mettant en évidence les interactions entre les cinq classes principales : **Bibliothèque**, **Livre**, **Membre**, **Maillon** et **ListeEmprunt**.



3.2. Description des classes

Notre application est structurée autour de cinq classes fondamentales, chacune ayant un rôle spécifique :

Classe1 : Livre

Rôle : Représente un livre de la bibliothèque.

Attributs : id_livre, titre, auteur, annee_publication, disponibilite.

Classe3 : Maillon

Rôle : Représente un emprunt individuel dans la liste chaînée.

Attributs : id_maillon, livre, membre, dateEmprunt, dateRetour, precedent, suivant.

C'est l'élément fondamental de notre structure de données.

Classe4 : ListeEmprunt

Rôle : Gère l'ensemble des emprunts.

Attributs : tete, queue, taille.

Méthodes clés :

- ajouterEnTete(),
- supprimerEmprunt(),
- rechercherLivre(),
- naviguerPrecedent(),
- naviguerSuivant(),
- compterEmprunts(),
- trierParDate().

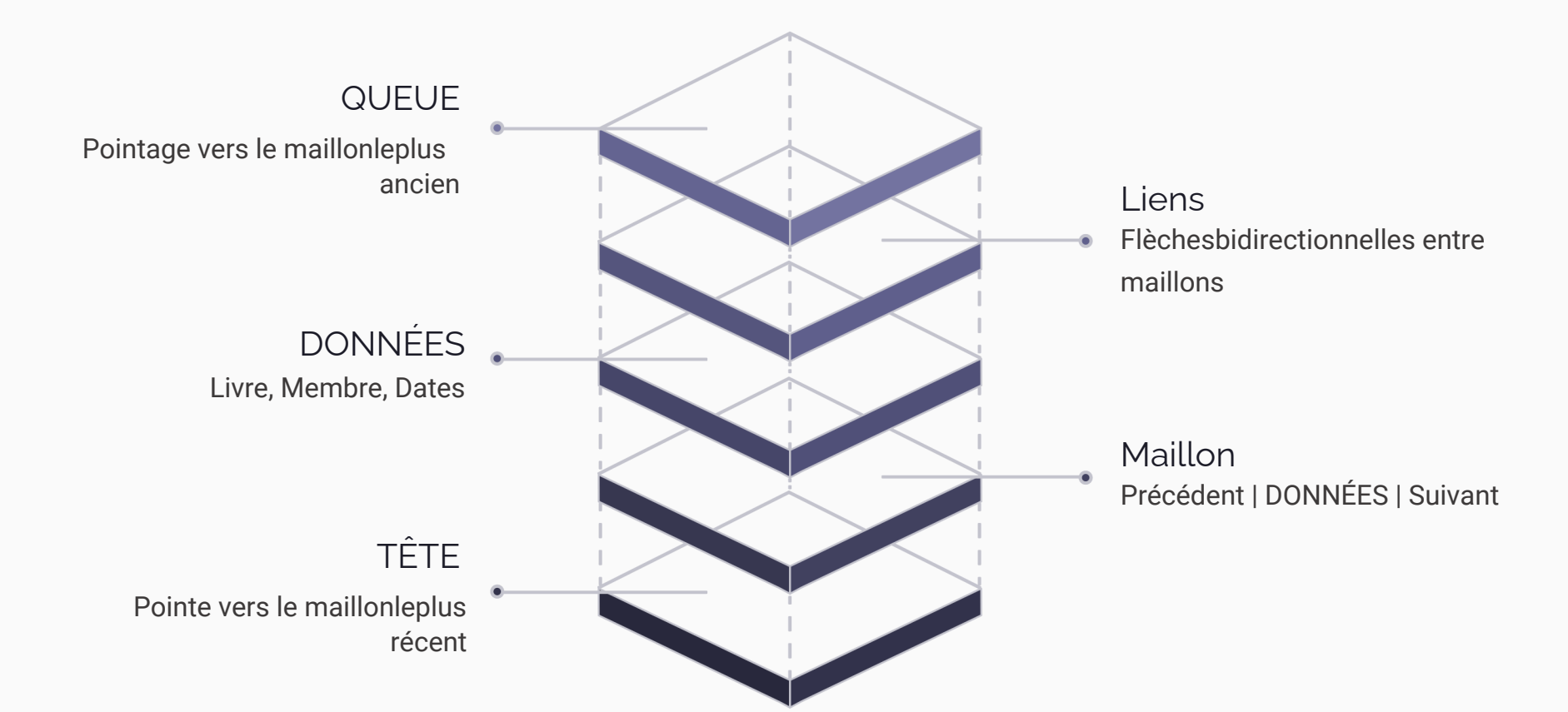
Classe5 : Bibliotheque

Rôle : Classe principale coordonnant toutes les opérations.

Attributs : catalogue (livres), membres, emprunts (ListeEmprunt).

3.3. Structure de la liste chaînée

La liste doublement chaînée est au cœur de notre système de gestion des emprunts. Chaque **Maillon** contient les données d'un emprunt (Livre, Membre, Dates) et deux pointeurs : precedent et suivant. Cela permet une navigation fluide dans les deux sens.



Exemple concret :

- Emprunt 1 (10/01/2025) : "Le Petit Prince" par Jean
- Emprunt 2 (12/01/2025) : "1984" par Marie
- Emprunt 3 (15/01/2025) : "L'Étranger" par Paul

Cette structure permet de naviguer de l'emprunt 1 → 2 → 3 (suivant) et de l'emprunt 3 → 2 → 1 (précédent), offrant une flexibilité essentielle pour l'historique.

4. RÉALISATION

4.1. Organisation du code

Notre projet est structuré de manière modulaire pour faciliter la maintenance et l'évolutivité. L'arborescence des fichiers est la suivante :



Cette séparation claire entre les `models/` (gestion des données et de la logique métier) et les `views/` (interfaces utilisateur) rend le code plus lisible, plus facile à déboguer et à modifier, en adhérant aux principes de la conception logicielle.

4.2. Les structures de données

Les algorithmes fondamentaux de notre application, basés sur la liste doublement chaînée, sont optimisés pour les opérations courantes :

1

1. Ajouter un emprunt en tête

Cette opération est **très rapide** avec une complexité de $O(1)$. Elle implique la création d'un nouveau maillon et la modification de quelques pointeurs (`tete`, `precedent`, `suivant`) sans parcourir la liste.

```
void ajouterEnTete(Livre* livre, Membre* membre, dates) {
    Maillon* nouveau = new Maillon(...);
    if (tete == NULL) { tete = nouveau; queue = nouveau; }
    else { nouveau->suivant = tete; tete->precedent = nouveau; tete = nouveau; }
}
```

2

2. Supprimer un emprunt

La suppression est de complexité $O(n)$, car elle nécessite de parcourir la liste pour localiser le maillon à supprimer. Une fois trouvé, les pointeurs adjacents sont réajustés.

```
void supprimerEmprunt(int id) {
    Maillon* courant = tete;
    while (courant != NULL) {
        if (courant->id == id) {
            if (courant->precedent != NULL) courant->precedent->suivant = courant->suivant;
            if (courant->suivant != NULL) courant->suivant->precedent = courant->precedent;
            delete courant; break;
        }
        courant = courant->suivant;
    }
}
```

3

3. Rechercher un livre

La recherche d'un livre a une complexité de $O(n)$. Elle implique un parcours séquentiel de la liste jusqu'à ce que le livre soit trouvé ou que la fin de la liste soit atteinte.

```
Maillon* rechercherLivre(string titre) {
    Maillon* courant = tete;
    while (courant != NULL) {
        if (courant->livre->titre == titre) { return courant; }
        courant = courant->suivant;
    }
    return NULL;
}
```

4

4. Trier les emprunts

Le tri des emprunts, par exemple par date, utilise un algorithme de tri à bulles, résultant en une complexité de $O(n^2)$, ce qui est **lent** pour de grandes listes.

```
void trierParDate() {
    bool exchange;
    do {
        exchange = false; Maillon* courant = tete;
        while (courant->suivant != NULL) {
            if (courant->date > courant->suivant->date) {
                echangerDonnees(courant, courant->suivant); exchange = true;
            }
            courant = courant->suivant;
        }
    } while (exchange);
}
```

Voici un récapitulatif des performances :

Ajouter en tête	$O(1)$	Très rapide
Supprimer	$O(n)$	Moyen
Rechercher	$O(n)$	Moyen
Naviguer $\longleftrightarrow$	$O(1)$	Très rapide
Trier	$O(n^2)$	Lent

4.3. Les interfaces graphiques

L'application est dotée de plusieurs interfaces graphiques, chacune dédiée à une fonctionnalité spécifique :

- 1

Page d'accueil

La première fenêtre affichée, présentant le logo de l'application et un bouton "Commencer" pour accéder au menu principal.
- 2

Page principale

Le menu central avec six boutons pour les fonctionnalités principales : enregistrer un emprunt, retourner un livre, rechercher, naviguer dans l'historique, voir les statistiques et trier le catalogue.
- 3

Fenêtre "Enregistrer un emprunt"

Un formulaire détaillé pour saisir le matricule du membre, choisir le livre, et définir les dates d'emprunt et de retour. Le système valide l'existence du membre et la disponibilité du livre avant d'ajouter l'emprunt.
- 4

Fenêtre "Retour de livre"

Une interface simple affichant la liste des livres actuellement empruntés, avec un bouton "Retourner" pour supprimer l'emprunt et rendre le livre disponible.
- 5

Fenêtre "Recherche"

Permet de saisir le titre d'un livre et d'afficher son statut (emprunté ou disponible) après recherche dans la liste.
- 6

Fenêtre "Navigation"

Affiche l'emprunt actuel et propose des boutons "← Précédent" et "Suivant →" pour parcourir l'historique via les pointeurs de la liste chaînée.

5. TESTS ET RÉSULTATS

5.1. Captures d'écran

Figure1 : Page d'accueil

Cette page est la première interaction de l'utilisateur avec l'application. Elle présente un design épuré, mettant en avant le logo de la bibliothèque et un bouton clair "Commencer" qui invite l'utilisateur à naviguer vers le menu principal. L'objectif est de fournir une entrée simple et intuitive dans le système.

Figure 2 : Page principale

Le menu principal est le centre de commande de l'application. Il est organisé autour de six boutons distincts, chacun correspondant à une fonctionnalité clé du système de gestion des emprunts. Cette disposition permet une navigation rapide et un accès direct aux opérations telles que l'enregistrement d'emprunts, les retours, la recherche, la navigation historique, les statistiques et le tri du catalogue. Le design est conçu pour être clair et fonctionnel, minimisant le temps d'apprentissage pour les utilisateurs.

Figure 3 : Formulaire d'emprunt

Le formulaire d'enregistrement d'emprunt est une interface détaillée qui permet aux bibliothécaires de saisir toutes les informations nécessaires pour un nouvel emprunt. Il comprend des champs pour le matricule du membre, une liste déroulante pour sélectionner le livre, et des calendriers pour spécifier les dates d'emprunt et de retour prévues. Un bouton "Valider" déclenche la vérification de l'existence du membre et de la disponibilité du livre, garantissant l'intégrité des données avant l'ajout de l'emprunt à la liste.

Figure 4 : Message de confirmation

Après la soumission réussie d'un formulaire d'emprunt, un message de confirmation s'affiche pour informer l'utilisateur que l'opération a été effectuée avec succès. Ce retour visuel est essentiel pour rassurer l'utilisateur et confirmer que l'emprunt a bien été enregistré dans le système. Le message est concis et clair, indiquant que le processus est terminé et que les données ont été mises à jour.

Figure 5 : Fenêtre de recherche

La fenêtre de recherche offre une fonctionnalité essentielle pour localiser rapidement des livres et vérifier leur statut. L'utilisateur peut saisir le titre d'un livre dans un champ dédié et lancer la recherche. Le système interroge la liste des emprunts et affiche les informations pertinentes sur le livre trouvé, y compris son statut actuel (disponible ou emprunté). Cette interface est conçue pour être rapide et efficace, fournissant des réponses immédiates aux requêtes des utilisateurs.